CS300

Week 3 · Agile Software Development

Agile Software Development

Elements of Software Engineering

Nguyễn Thị Minh Tuyền · Faculty of Information Technology, HCMUS

Adapted from slides of Ian Sommerville

Topics Covered

1. Agile methods
2. Extreme programming
3. Agile project management

Topics Covered

1. **Agile methods**
2. Extreme programming
3. Agile project management

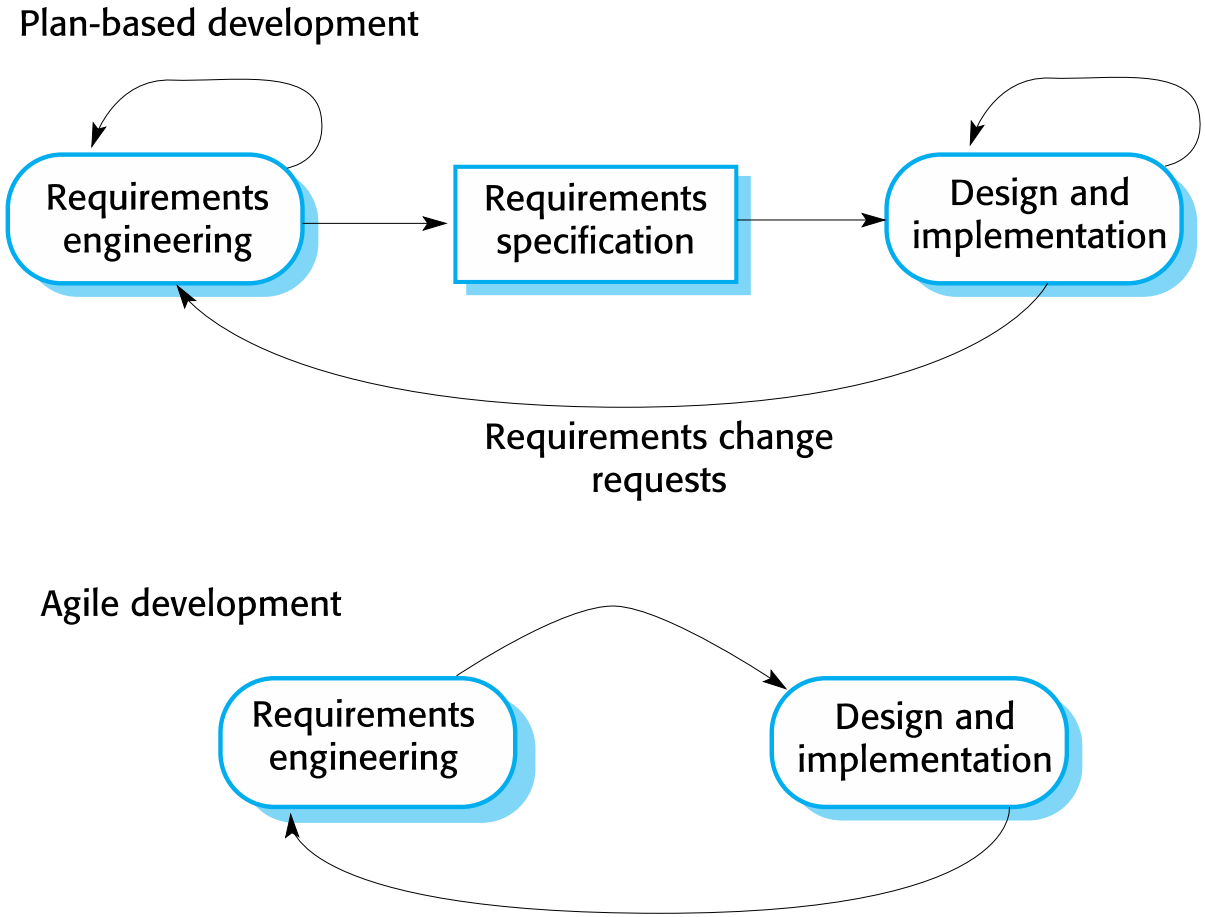
Rapid Software Development

- Rapid development and delivery is now often the most important requirement for software systems.
  - Businesses operate in a fast-changing environment and it is practically impossible to produce a set of stable software requirements.
  - Software has to evolve quickly to reflect changing business needs.
- Plan-driven development is essential for some types of system but does not meet these business needs.
- Agile development methods emerged in the late 1990s whose aim was to radically reduce the delivery time for working software systems.

Agile Development

- Program specification, design and implementation are inter-leaved.
- The system is developed as a series of versions or increments with stakeholders involved in version specification and evaluation.
- Frequent delivery of new versions for evaluation.
- Extensive tool support (e.g. automated testing tools) used to support development.
- Minimal documentation – focus on working code.

Plan-driven and Agile Development



Plan-driven and Agile Development

- **Plan-driven development**
  - A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance.
  - Not necessarily waterfall model – plan-driven, incremental development is possible.
  - Iteration occurs within activities.
- **Agile development**
  - Specification, design, implementation and testing are inter-leaved and the outputs from the development process are decided through a process of negotiation during the software development process.

Agile Methods

- Dissatisfaction with the overheads involved in software design methods of the 1980s and 1990s led to the creation of agile methods. These methods:
  - Focus on the code rather than the design.
  - Are based on an iterative approach to software development.
  - Are intended to deliver working software quickly and evolve this quickly to meet changing requirements.
- The aim of agile methods is to reduce overheads in the software process (e.g. by limiting documentation) and to be able to respond quickly to changing requirements without excessive rework.

Agile Manifesto

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

- **Individuals and interactions** over processes and tools
- **Working software** over comprehensive documentation
- **Customer collaboration** over contract negotiation
- **Responding to change** over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Principles of Agile Methods

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is to provide and to prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.

Agile Method Applicability

- **Product development**
  - where a software company is developing a small or medium-sized product for sale.
- **Custom system development within an organization**
  - where there is a clear commitment from the customer to become involved in the development process and where there are not a lot of external rules and regulations that affect the software.

Because of their focus on small, tightly-integrated teams, there are problems in scaling agile methods to large systems.

Topics Covered

1. Agile methods
2. **Extreme programming**
3. Agile project management

Agile Methods

- Agile Modeling
- Agile Unified Process (AUP)
- Dynamic Systems Development Method (DSDM)
- Essential Unified Process (EssUP)
- **Extreme Programming (XP)**
- Feature Driven Development (FDD)
- Open Unified Process (OpenUP)
- **Scrum**
- Velocity tracking

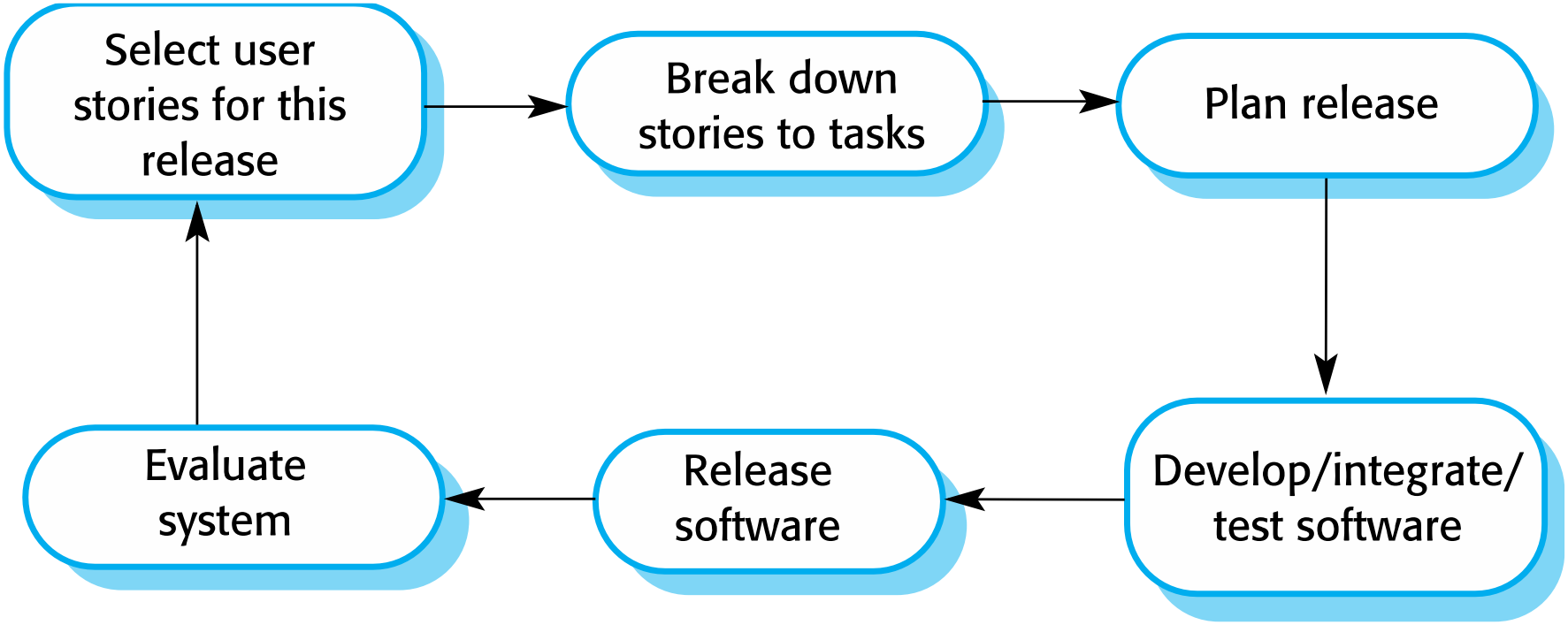
Extreme Programming

- A very influential agile method, developed in the late 1990s, that introduced a range of agile development techniques.
- Extreme Programming (XP) takes an ‘extreme’ approach to iterative development.
  - New versions may be built **several times per day**;
  - Increments are delivered to customers **every 2 weeks**;
  - **All tests must be run for every build** and the build is only accepted if tests run successfully.

XP and Agile Principles

- **Incremental development** is supported through small, frequent system releases.
- **Customer involvement** means full-time customer engagement with the team.
- **People not process** through pair programming, collective ownership and a process that avoids long working hours.
- **Change supported** through regular system releases.
- **Maintaining simplicity** through constant refactoring of code.

The Extreme Programming Release Cycle



From user stories to an evaluated release – repeated every increment.

Extreme Programming Practices (1/2)

Principle or practice	Description
Incremental planning	Requirements are recorded on **story cards** and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development 'Tasks'.
Small releases	The **minimal useful set of functionality that provides business value is developed first**. Releases of the system are frequent and incrementally add functionality to the first release.
Simple design	Enough design is carried out to **meet the current requirements and no more**.
Test-first development	**An automated unit test framework is used** to write tests for a new piece of functionality before that functionality itself is implemented.
Refactoring	All developers are expected to refactor the code continuously as soon as possible code improvements are found. This **keeps the code simple and maintainable**.

Extreme Programming Practices (2/2)

Principle or practice	Description
Pair programming	Developers **work in pairs**, checking each other's work and providing the support to always do a good job.
Collective ownership	The pairs of developers work on all areas of the system, so that no islands of expertise develop and all the developers take responsibility for all of the code. **Anyone can change anything.**
Continuous integration	As soon as the work on a task is complete, **it is integrated into the whole system**. After any such integration, all the unit tests in the system must pass.
Sustainable pace	**Large amounts of overtime are not considered acceptable** as the net effect is often to reduce code quality and medium term productivity.
On-site customer	A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In an extreme programming process, the **customer is a member of the development team** and is responsible for **bringing system requirements to the team** for implementation.

Influential XP Practices

- Extreme programming has a technical focus and is not easy to integrate with management practice in most organizations.
- **Key practices**
  - User stories for specification
  - Refactoring
  - Test-first development
  - Pair programming

User Stories for Requirements

- In XP, a customer or user is part of the XP team and is responsible for making decisions on requirements.
- User requirements are expressed as scenarios or user stories.
- These are written on cards and the development team break them down into implementation tasks. These tasks are the basis of schedule and cost estimates.
- The customer chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.

A ‘Prescribing Medication’ Story

Prescribing medication

The record of the patient must be open for input. Click on the medication field and select either ‘current medication’, ‘new medication’ or ‘formulary’.

If you select ‘current medication’, you will be asked to check the dose; If you wish to change the dose, enter the new dose then confirm the prescription.

If you choose, ‘new medication’, the system assumes that you know which medication you wish to prescribe. Type the first few letters of the drug name. You will then see a list of possible drugs starting with these letters. Choose the required medication. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

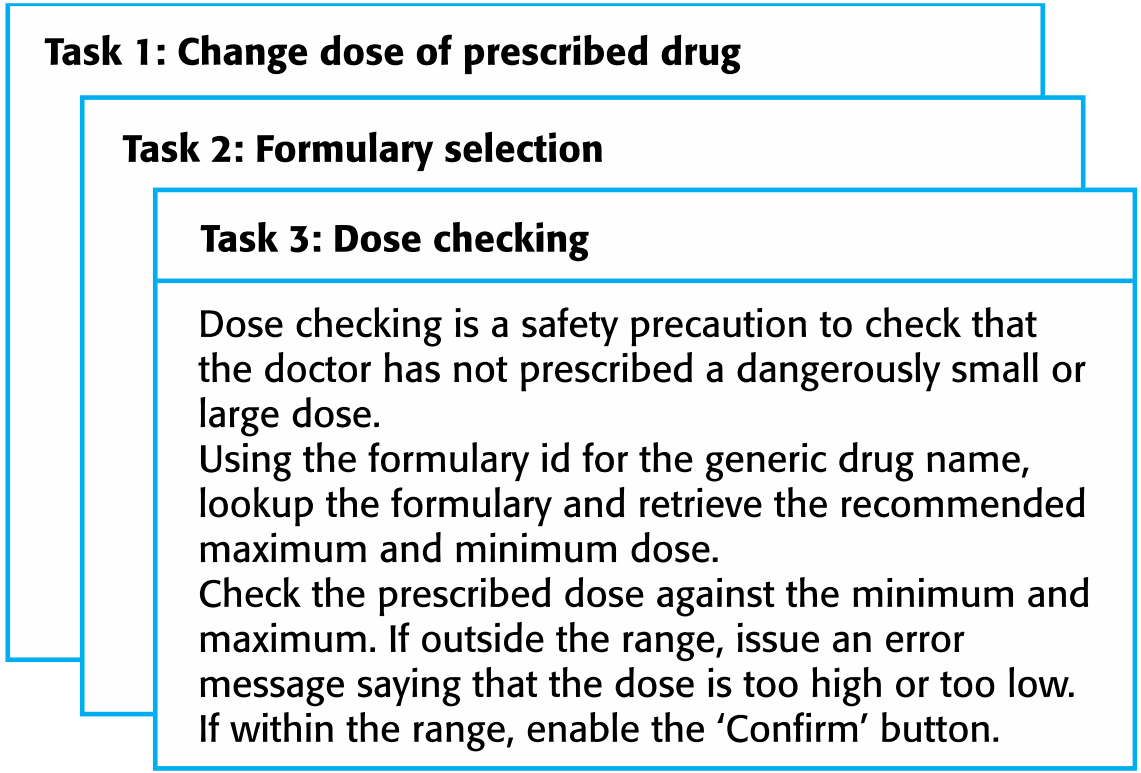
If you choose ‘formulary’, you will be presented with a search box for the approved formulary. Search for the drug required then select it. You will then be asked to check that the medication you have selected is correct. Enter the dose then confirm the prescription.

In all cases, the system will check that the dose is within the approved range and will ask you to change it if it is outside the range of recommended doses.

After you have confirmed the prescription, it will be displayed for checking. Either click ‘OK’ or ‘Change’. If you click ‘OK’, your prescription will be recorded on the audit database. If you click ‘Change’, you reenter the ‘Prescribing medication’ process.

A user story card from the Mentcare system.

Examples of Task Cards for Prescribing Medication



The story is broken down into implementation tasks.

Refactoring

- Programming team look for possible software improvements and make these improvements even where there is no immediate need for them.
- This improves the understandability of the software and so reduces the need for documentation.
- Changes are easier to make because the code is well-structured and clear.
- However, some changes require architecture refactoring and this is much more expensive.

Examples of Refactoring

- Re-organization of a class hierarchy to remove duplicate code.
- Tidying up and renaming attributes and methods to make them easier to understand.
- Replacement of inline code with calls to methods that have been included in a program library.

XP and Change

- Conventional wisdom in software engineering is to design for change. It is worth spending time and effort anticipating changes as this reduces costs later in the life cycle.
- XP, however, maintains that this is not worthwhile as changes cannot be reliably anticipated.
- Rather, it proposes **constant code improvement** (refactoring) to make changes easier when they have to be implemented.

Test-first Development

- Testing is central to XP and XP has developed an approach where the program is tested after every change has been made.
- **XP testing features:**
  - Test-first development.
  - Incremental test development from scenarios.
  - User involvement in test development and validation.
  - Automated test harnesses are used to run all component tests each time that a new release is built.

Test-driven Development

- Writing tests before code clarifies the requirements to be implemented.
- Tests are written as programs rather than data so that they can be executed automatically. The test includes a check that it has executed correctly.
  - Usually relies on a testing framework such as JUnit.
- All previous and new tests are run automatically when new functionality is added, thus checking that the new functionality has not introduced errors.

Customer Involvement

- The role of the customer in the testing process is to help **develop acceptance tests** for the stories that are to be implemented in the next release of the system.
- The customer who is part of the team **writes tests** as development proceeds. All new code is therefore validated to ensure that it is what the customer needs.
- However, people adopting the customer role have limited time available and so cannot work full-time with the development team.
    - They may feel that providing the requirements was enough of a contribution and so may be reluctant to get involved in the testing process.

Test Case Description for Dose Checking

Test 4: Dose checking

Input:
1. A number in mg representing a single dose of the drug.
2. A number representing the number of single doses per day.

Tests:
1. Test for inputs where the single dose is correct but the frequency is too high.
2. Test for inputs where the single dose is too high and too low.
3. Test for inputs where the single dose * frequency is too high and too low.
4. Test for inputs where single dose * frequency is in the permitted range.

Output:
OK or error message indicating that the dose is outside the safe range.

Test Automation

- Tests are written as executable components before the task is implemented.
  - These testing components should be stand-alone, should simulate the submission of input to be tested and should check that the result meets the output specification.
  - An automated test framework (e.g. JUnit) is a system that makes it easy to write executable tests and submit a set of tests for execution.
- There is always a set of tests that can be quickly and easily executed.
  - Whenever any functionality is added to the system, the tests can be run and problems that the new code has introduced can be caught immediately.

Problems with Test-first Development

- Programmers prefer programming to testing and sometimes they take shortcuts when writing tests.
  - For example, they may write incomplete tests that do not check for all possible exceptions that may occur.
- Some tests can be very difficult to write incrementally.
  - For example, in a complex user interface, it is often difficult to write unit tests for the code that implements the ‘display logic’ and workflow between screens.
- It is difficult to judge the completeness of a set of tests. Although you may have a lot of system tests, your test set may not provide complete coverage.

Pair Programming

- Programmers sit together at the same workstation to develop the software.
- Pairs are **created dynamically** so that all team members work with each other during the development process.
- The **sharing of knowledge** that happens during pair programming is very important as it **reduces the overall risks** to a project when team members leave.
- Pair programming is not necessarily inefficient and there is evidence that a pair working together is more efficient than 2 programmers working separately.

Advantages of Pair Programming

- It supports the idea of collective ownership and responsibility for the system.
  - Individuals are not held responsible for problems with the code. Instead, the team has collective responsibility for resolving these problems.
- It acts as an informal review process because each line of code is looked at by at least two people.
- It helps support refactoring, which is a process of software improvement.
  - Where pair programming and collective ownership are used, others benefit immediately from the refactoring so they are likely to support the process.

Topics Covered

1. Agile methods
2. Extreme programming
3. **Agile project management**

Agile Project Management

- The principal responsibility of software project managers is to manage the project so that the **software is delivered on time and within the planned budget** for the project.
  - The standard approach to project management is plan-driven.
  - Managers draw up a plan for the project showing what should be delivered, when it should be delivered and who will work on the development of the project deliverables.
- Agile project management requires a different approach, which is adapted to incremental development and the particular strengths of agile methods.

Scrum

- Scrum is an agile method that focuses on managing iterative development rather than specific agile practices.
- **There are three phases in Scrum:**
  - The initial phase.
  - A series of sprint cycles.
  - The project closure phase.

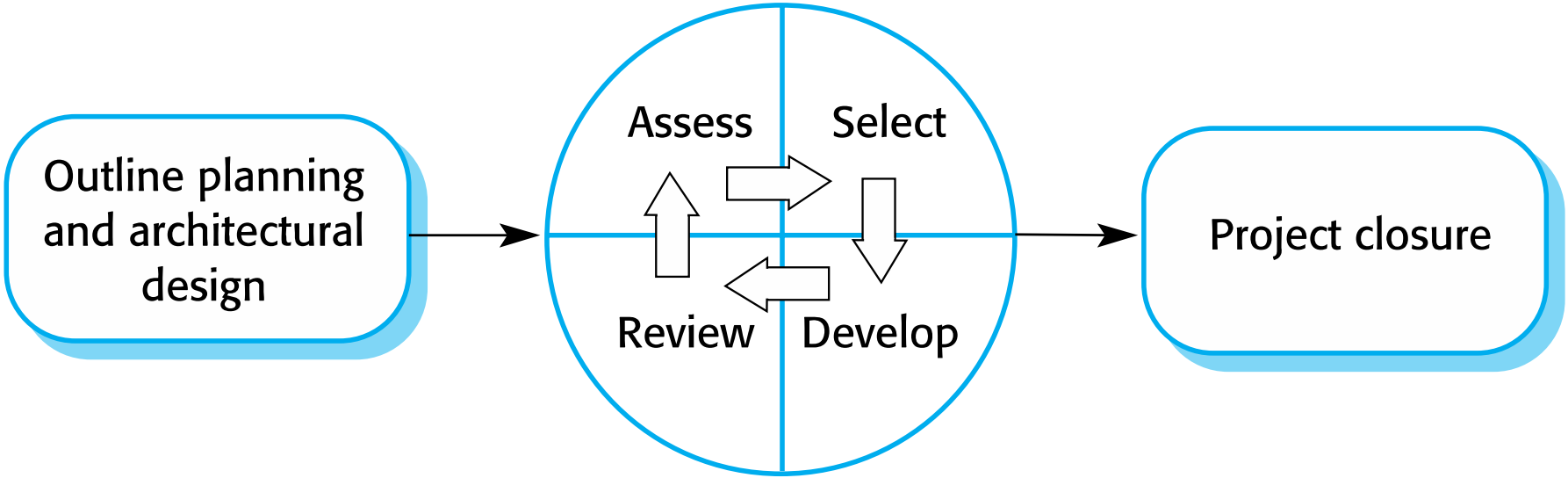
Scrum Terminology (1/2)

Scrum term	Definition
Development team	A self-organizing group of software developers, which should be no more than 7 people. They are responsible for developing the software and other essential project documents.
Potentially shippable product increment	The software increment that is delivered from a sprint. The idea is that this should be ‘potentially shippable’ which means that it is in a finished state and no further work, such as testing, is needed to incorporate it into the final product. In practice, this is not always achievable.
Product backlog	This is a list of ‘to do’ items which the Scrum team must tackle. They may be feature definitions for the software, software requirements, user stories or descriptions of supplementary tasks that are needed, such as architecture definition or user documentation.
Product owner	An individual (or possibly a small group) whose job is to identify product features or requirements, prioritize these for development and continuously review the product backlog to ensure that the project continues to meet critical business needs. The Product Owner can be a customer but might also be a product manager in a software company or other stakeholder representative.

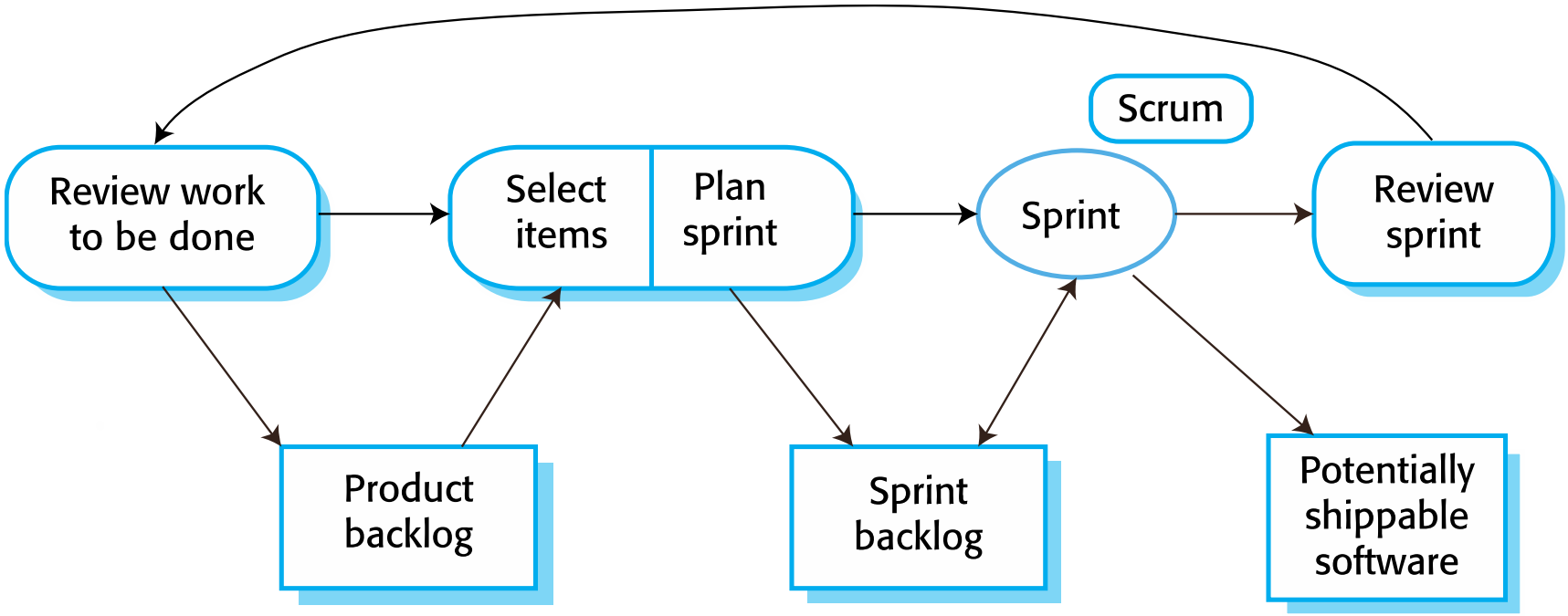
Scrum Terminology (2/2)

Scrum term	Definition
Scrum	A daily meeting of the Scrum team that reviews progress and prioritizes work to be done that day. Ideally, this should be a short face-to-face meeting that includes the whole team.
ScrumMaster	The ScrumMaster is responsible for ensuring that the Scrum process is followed and guides the team in the effective use of Scrum. He or she is responsible for interfacing with the rest of the company and for ensuring that the Scrum team is not diverted by outside interference. The Scrum developers are adamant that the ScrumMaster should not be thought of as a project manager. Others, however, may not always find it easy to see the difference.
Sprint	A development iteration. Sprints are usually 2–4 weeks long.
Velocity	An estimate of how much product backlog effort that a team can cover in a single sprint. Understanding a team's velocity helps them estimate what can be covered in a sprint and provides a basis for measuring improving performance.

Scrum Process



Scrum Sprint Cycle



Sprint Cycle

- **Fixed length**, normally 2–4 weeks. They correspond to the development of a release of the system in XP.
- The starting point for planning is the **product backlog**, which is the list of work to be done on the project.
- The selection phase involves all of the project team who work with the customer to select the features and functionality to be developed during the sprint.
- Once these are agreed, the team organize themselves to develop the software.
    - During this stage the team is isolated from the customer and the organization, with all communications channelled through the so-called ‘Scrum master’.
- The **role of the Scrum master is to protect the development team from external distractions**.
- At the end of the sprint, the work done is reviewed and presented to stakeholders. The next sprint cycle then begins.

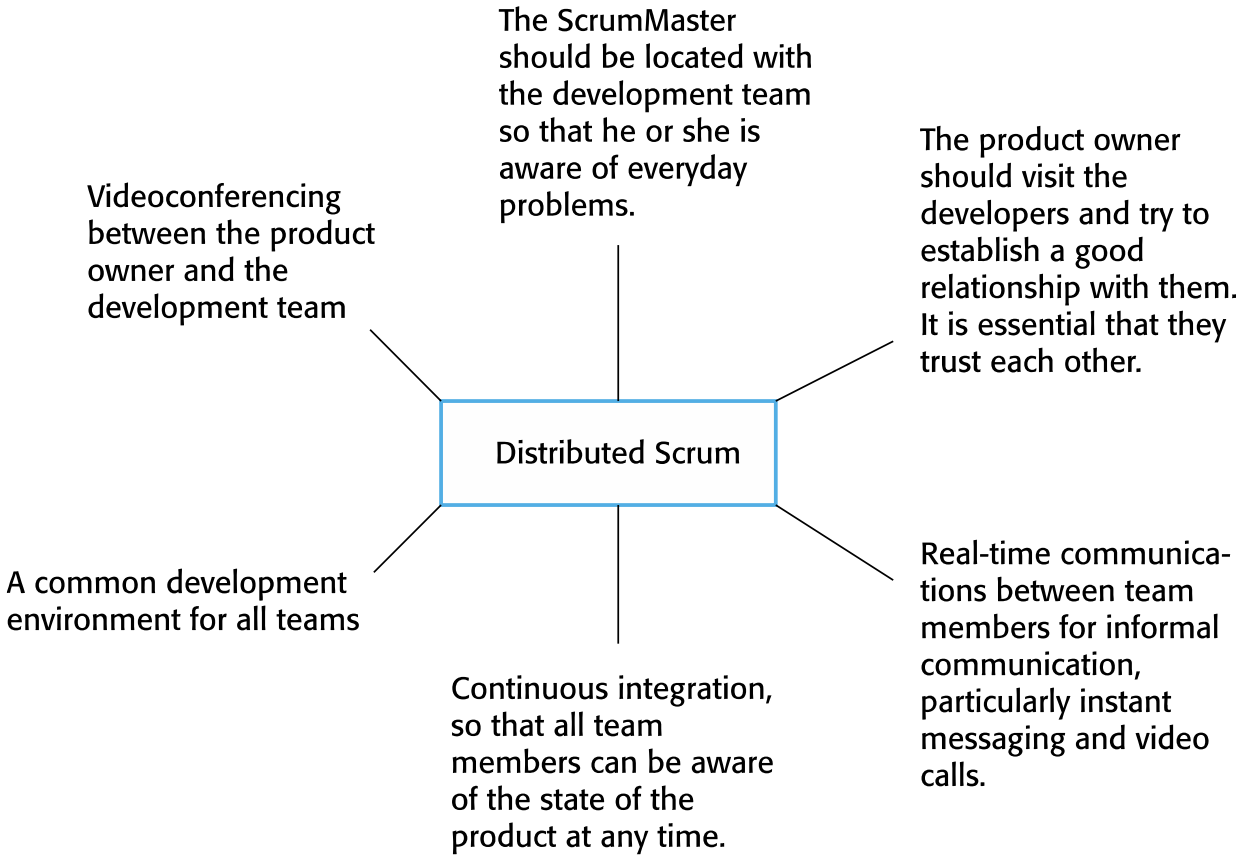
Teamwork in Scrum

- The '**Scrum master**' is a facilitator who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside of the team.
- The whole team attends short daily meetings where all team members share information, describe their progress since the last meeting, problems that have arisen and what is planned for the following day.
    - This means that everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them.

Scrum Benefits

- The product is broken down into a set of manageable and understandable chunks.
- Unstable requirements do not hold up progress.
- The whole team have visibility of everything and consequently team communication is improved.
- Customers see on-time delivery of increments and gain feedback on how the product works.
- Trust between customers and developers is established and a positive culture is created in which everyone expects the project to succeed.

Distributed Scrum



Questions?

Reading: Sommerville, Software Engineering, 10th Ed — Chapter 3

Nguyễn Thị Minh Tuyền · Faculty of IT, HCMUS

CS300 · Elements Software Engineering · Week 3